

SMART MCQ SOLVER

Detailed Project Report on Deep Learning and Generative AI

Submitted by

Vikash Kumar

IIT Madras — BS Degree Program in Data Science and Applications

Diploma Level — Deep Learning & Generative AI Project

September 2026

Abstract

The Smart MCQ Solver is an end-to-end Natural Language Processing and Deep Learning project designed to predict the three most probable answers for multiple-choice science questions. The project studies the problem from several levels of machine learning complexity. It begins with exploratory data analysis and classical text representation using TF-IDF, progresses to an Artificial Neural Network (ANN) implemented in PyTorch, and then investigates transformer-based language modeling with DistilBERT. Additional milestone work covers attention mechanisms, contextual sentence embeddings, zero-shot classification, generative models, and parameter-efficient fine-tuning using LoRA. The project also includes experiment tracking with Weights & Biases, model persistence, automatic submission generation, and an interactive Gradio deployment on Hugging Face Spaces. The final deployed application uses the ANN with a saved TF-IDF vectorizer and returns a ranked Top-3 prediction together with confidence information.

1. Introduction

Multiple-choice question answering appears simple because the output is one of a small number of choices. However, a useful automated solver must understand the relationship between the question and all candidate options. The correct option may use different words from the question, contain domain-specific terminology, or require understanding a concept rather than matching an exact phrase. This makes MCQ solving a useful Natural Language Processing task.

The Smart MCQ Solver project was developed to study this problem through a progression of machine learning and deep learning techniques. The implementation does not depend on a single algorithm. Instead, it establishes a classical baseline, builds a neural classifier, experiments with a transformer model, and investigates more advanced NLP concepts through milestone notebooks.

The final objective is a ranking task. For every question, five options are available: A, B, C, D, and E. The system produces the three options with the highest predicted probabilities. This Top-3 formulation is important because the competition evaluation rewards the position of the correct answer in the ranked list rather than requiring only a single class prediction.

1.1 Problem Statement

Given a science-related multiple-choice question and five possible answers, develop a machine learning/deep learning pipeline that can identify and rank the most probable answer choices. The system must accept the question and all five options, convert the text into numerical features, predict probabilities for the five classes, and return the Top-3 labels in ranked order.

1.2 Objectives

- Understand the structure and quality of the MCQ dataset through EDA.
- Clean text and construct a consistent input representation.
- Establish a TF-IDF baseline.
- Develop a PyTorch ANN.
- Fine-tune DistilBERT.
- Study attention, embeddings, zero-shot classification, and generation.
- Experiment with LoRA.
- Track experiments with W&B.;
- Save artifacts and generate submissions.
- Deploy an interactive Top-3 solver.

2. Dataset and Competition Formulation

The project notebooks work with a training dataset of 2,000 examples and a test dataset of 500 examples. Each training example contains an identifier, a question prompt, five answer choices (A–E), and a target answer label. The test set contains the identifier, prompt, and five options but does not contain the target.

The task is treated as five-class classification during training. Labels A–E are mapped to integer IDs 0–4 and mapped back after prediction. The final output contains the three highest-scoring answer choices.

Dataset	Size	Main fields
Training	2,000	id, prompt, A, B, C, D, E, answer
Test	500	id, prompt, A, B, C, D, E
Submission	500 predictions	id + Top-3 labels

3. Exploratory Data Analysis

EDA checks dataset shape, data types, missing values, duplicate rows, target distribution, and text lengths. The inspected main fields did not contain missing values. Question and option lengths were examined because they influence both TF-IDF behavior and transformer token limits. Visual and qualitative inspection included boxplots, sample questions, and word-frequency exploration.

- Shape and preview
- Missing-value checks
- Duplicate detection
- Class distribution
- Question/option lengths
- Boxplots
- Sample questions
- Word-cloud / frequency analysis

4. Data Preprocessing and Feature Construction

4.1 Input Formatting

The question and all five options are combined into a structured text sequence. The deployment uses the same representation as training:

Question:

A:
B:
C:
D:
E:

Keeping the representation identical between training and inference is essential for reproducibility.

4.2 Label Encoding

The answer labels A–E are converted into numeric class IDs 0–4. The mapping is saved so predictions can be converted back into human-readable labels.

4.3 TF-IDF

TF-IDF assigns numerical importance to words and n-grams. The project uses lowercase text, Unicode accent stripping, up to 150,000 features, n-grams from 1 to 3, min_df=2, max_df=0.95, sublinear TF, smoothed IDF, and L2 normalization. TF-IDF is efficient and interpretable but mainly captures lexical patterns.

4.4 Transformer Tokenization

DistilBERT tokenization uses a maximum length of 256 tokens with truncation and max-length padding. Input IDs and attention masks are supplied to the transformer.

5. Project Architecture and Workflow

The workflow consists of EDA, preprocessing, feature extraction, model training, validation, Top-3 ranking, submission generation, artifact saving, and deployment.

Stage	Technology	Output
EDA	Pandas/NumPy/Matplotlib	Data understanding
Text features	TF-IDF	Sparse vectors
Neural model	PyTorch ANN	Five-class logits
Transformer	DistilBERT	Contextual logits
Classical	Logistic Regression	Probabilities
Tracking	W&B	Experiment metrics
Deployment	Gradio + Spaces	Interactive solver

6. Milestone 0 — Foundation

This milestone establishes the data contract, five-class formulation, label encoding, text formatting, and validation split used by later models.

7. Milestone 1 — TF-IDF and Similarity

TF-IDF vectors can be compared using cosine similarity: $\cos(A,B) = (A \cdot B) / (||A|| ||B||)$. This gives a transparent lexical ranking baseline, but it can fail when the correct answer uses different wording or when a distractor shares many surface words.

8. Milestone 2 — Attention and Contextual Embeddings

The project introduces self-attention and contextual representations. Sentence-transformers/all-MiniLM-L6-v2 is used for semantic embeddings. One recorded example gives cosine similarity of approximately 0.7658 between a prompt and Option B. The milestone also explores zero-shot classification and a small generative experiment.

9. Milestone 3 — Transformer NLP

DistilBERT is used as a pretrained contextual encoder. The tokenizer creates 256-token sequences and attention masks. A five-class classification head is trained on the answer labels.

Parameter	Setting
Base model	distilbert-base-uncased
Labels	5
Max length	256
Batch size	16
Learning rate	1e-5
Prediction	Top-3

10. Milestone 4 — LoRA / PEFT

The LoRA experiment records 109,778,690 total parameters and 295,681 trainable parameters, approximately 0.2693% trainable. This demonstrates parameter-efficient adaptation of a large pretrained model.

11. Milestone 5 — MAP@3

MAP@3 evaluates the quality and ordering of the first three predictions. One recorded milestone output reports MAP@3 = 0.3267. This is a milestone-specific result and should not be treated as the final deployed model's universal score.

12. Model 1 — Artificial Neural Network

The final deployed model is a PyTorch ANN operating on TF-IDF features. It contains Linear(input $\rightarrow$ 512), ReLU, Dropout(0.3), Linear(512 $\rightarrow$ 256), ReLU, Dropout(0.3), Linear(256 $\rightarrow$ 128), ReLU, Dropout(0.3), and Linear(128 $\rightarrow$ 5). CrossEntropyLoss is used for training and Adam for optimization. At inference, softmax produces probabilities.

Layer	Input → Output	Operation
Input	TF-IDF → 512	Linear + ReLU + Dropout
Hidden 2	512 → 256	Linear + ReLU + Dropout
Hidden 3	256 → 128	Linear + ReLU + Dropout
Output	128 → 5	Linear logits

12.1 Training Results

Epoch	Train	Validation
1	0.7963	0.0601
2	0.1016	0.0289
3	0.0421	0.0000
4	0.0363	0.0000
5	0.0000	0.0000
6	0.0388	0.0000
7	0.0066	0.0000
8	0.0190	0.0000
9	0.0572	0.0872
10	0.0341	0.0000

The loss decreases sharply, showing that the ANN fits the learned representation effectively. The temporary validation increase at epoch 9 also demonstrates why validation monitoring is useful. Because the dataset is small, additional independent evaluation is still desirable.

13. Model 2 — DistilBERT

DistilBERT receives token IDs and attention masks, computes classification loss, and updates pretrained parameters during fine-tuning. The training loop includes gradient clipping. Validation records loss, accuracy, and Macro F1, and the best checkpoint is saved according to validation Macro F1.

14. Model 3 — Logistic Regression and Ensemble

The classical model uses TF-IDF with 150,000 features and 1–3 word n-grams. Logistic Regression uses C=4, SAGA, L2 regularization, max_iter=6000, and random_state=42. Its probability output is combined with LightGBM:

$$P = w \cdot P_{LR} + (1-w) \cdot P_{LGBM}$$

Weights are searched from 0.10 to 0.95 in 0.05 increments using validation Macro F1.

15. Experiment Tracking

Weights & Biases is used to organize experiments and record training loss, validation loss, accuracy, Macro F1, and learning curves. This supports reproducibility and model comparison.

16. Submission Generation

The test set is transformed using the learned preprocessing artifacts. Top-3 class indices are extracted, mapped to A–E, and written into submission.csv. The notebook generates 500 predictions.

17. Final Deployment — Gradio

The deployment application loads best_ann.pt, tfidf_vectorizer.pkl, and id2label.pkl. It constructs the same ANN architecture, loads the state dictionary, switches to evaluation mode, formats the user's question and options, vectorizes the text, computes logits, applies softmax, ranks the three largest probabilities, and displays the result.

17.1 Prediction Flow

- Enter question and five options.
- Validate that fields are not empty.
- Format text consistently.
- Transform using saved TF-IDF.
- Run ANN inference.
- Convert logits to probabilities.
- Select Top-3.
- Map IDs to A–E.
- Display confidence and ranking.

17.2 Interface

The application uses Gradio 5.43.1 and is configured for Python 3.12 and Hugging Face Spaces. The interface contains a highlighted top result, detailed Top-3 cards, confidence information, and probability visualization.

18. Project File Structure

Path	Purpose
notebooks/01_EDA.ipynb	EDA
notebooks/milestone-0.ipynb	Foundation
notebooks/milestone-1.ipynb	TF-IDF / similarity
notebooks/milestone-2.ipynb	Attention / embeddings
notebooks/milestone-3.ipynb	Transformer experiments
notebooks/milestone-4.ipynb	LoRA / PEFT
notebooks/milestone-5.ipynb	Evaluation
kaggle_notebook.ipynb	Main training/submission
smart-mcq-ann/model.py	ANN architecture
smart-mcq-ann/app.py	Gradio inference
smart-mcq-ann/*.pkl	Saved preprocessing artifacts

19. Technologies Used

Technology	Role
Python	Programming
Pandas / NumPy	Data handling
Matplotlib / Seaborn	EDA
Scikit-learn	TF-IDF / Logistic Regression
PyTorch	ANN / deep learning
Hugging Face	Transformers
Sentence Transformers	Semantic embeddings
W&B	Experiment tracking
Gradio	Deployment UI
Hugging Face Spaces	Hosting

20. Limitations

The dataset is relatively small for large transformer fine-tuning. TF-IDF is lexical and may miss semantic relationships. Transformer training is computationally heavier. Very low validation loss on a small dataset does not automatically establish broad generalization. Some science MCQs require domain knowledge and reasoning beyond surface text.

21. Future Improvements

- Stronger pretrained encoders.
- Cross-validation and repeated splits.
- Systematic hyperparameter optimization.
- Lexical + neural + transformer ensemble.
- Retrieval from trusted science textbooks.
- Retrieval-augmented explanations.
- Probability calibration.
- Larger independent evaluation set.
- Batch prediction and API deployment.
- Versioned model/preprocessing artifacts.

22. Conclusion

The Smart MCQ Solver is a complete ML/DL/NLP workflow covering EDA, preprocessing, TF-IDF, ANN training, transformer fine-tuning, contextual embeddings, LoRA, evaluation, experiment tracking, submission generation, and deployment. The final Gradio application demonstrates how a trained model can be converted into a usable question-answering interface. The project also provides a foundation for future retrieval-augmented MCQ solving using trusted science resources.

23. References

- PyTorch documentation.
- Hugging Face Transformers documentation.
- Scikit-learn documentation.
- Sentence Transformers documentation.
- Weights & Biases documentation.
- Gradio documentation.
- Hugging Face Spaces documentation.
- Smart MCQ Solver Challenge materials.
- Project notebooks and source code.